

机器学习

第1章 机器学习概述

第2章 线性回归及最大熵模型

一、线性回归

最小二乘法

梯度下降法

二、逻辑回归

第3章 K最近邻算法

第4章 决策树算法

1、ID3算法

2、C4.5算法

3、C5.0算法

4、CART算法

算法关键问题：

决策树剪枝

代码使用

第5章 朴素贝叶斯算法

第6章 支持向量机

第七章 集成学习

第9章 PCA降维

第10章 聚类算法

第12章 卷积神经网络

第15章 BP神经网络

第1章 机器学习概述

构建一个完整的机器学习算法需要三要素：数据，模型，性能 度量指标

在训练之前调整设置的参数，称为超参数

分类问题以离散随即变量或者离散随机变量的概率分布作为预测输出，回归问题以连续变量作为预测输出

根据机器学习是否可用于生成新数据，可以将机器学习模型分为生成模型和判别模型。生成模型学习的是样本输入与标签的联合概率分布，所以可以从生成模型的联合概率分布中进行采用，从而生成新的数据，而判别模型只是一个条件概率分布模型，只能对输入进行判定。

生成模型：朴素，高斯混合模型，生成对抗网络

判别模型：逻辑回归，决策树，svm，神经网络，knn

人工智能>机器学习>深度学习

1、分类（Classification）：使用计算机学习出的模型进行预测得到的是**离散值**。

二分类：只涉及两个类别的分类任务，其中一个类为正类（Positive Class），另一个类为负类（Negative Class），如是猫、不是猫两类。

多分类（Multi-class Classification）：涉及多个类别的分类任务。

2、回归（Regression）：使用计算机学习出的模型进行预测得到的是**连续值**。

分类和回归的区别：

输出不同：分类问题输出的是物体所属的**类别**，回归问题输出的是物体的**值**；分类问题输出的值是**定性**的，回归问题输出的值是**定量**的。

目的不同：分类的目的是为了寻找决策边界；回归问题是为了找到最优拟合。

结果不同：分类的结果没有逼近，对错分明；回归是对真实值的一种逼近预测

泛化能力：学习到的模型适用于新样本的能力。机器学习的目标是使得学习到的模型能很好得适用于新样本，而不是仅仅在训练样本上适用。

机器学习的一般流程：

- 1、数据收集和预处理：爬虫，API，数据库 | 数据清洗，数据归一化/标准化
- 2、特征工程：特征构建、特征提取、特征选择
- 3、构建模型：① 建立训练数据集和测试数据集② 选择机器学习算法③模型调优④模型融合（多个模型）
- 4、评估和预测：



机器学习分类：

- 监督学习：分类和回归任务，决策树，贝叶斯模型，支持向量机，深度学习

- 无监督学习：聚类算法
- 半监督学习：没有标识的远大于被标识的数据数量
- 强化学习：基于与环境的交互进行学习，输入数据直接反馈到模型，模型必须作出调整

常见分类算法：支持向量机，神经网络，朴素贝叶斯，贝叶斯网络，k-最近邻

深度学习包括：卷积神经网络CNN，深度神经网络DNN，循环神经网络RNN，对抗神经网络GAN

当训练损失较大，说明模型不能对数据进行很好的拟合，称为欠拟合

当训练误差小而明显低于泛化误差是，称这种情况为过拟合

对于欠拟合的情况，通常是由于模型本身不能对训练集进行拟合或者训练迭代次数太少。在图 1-1 中，线性模型不能近似拟合二次函数。解决欠拟合的主要方法是对模型进行改进、设计新的模型重新训练、增加训练过程的迭代次数等。对于过拟合的情况，往往是由于数据量太少或者模型太过复杂导致，可以通过增加训练数据量、对模型进行裁剪、正则化等方式来缓解。

过拟合的处理：

- 1.获得更多的训练数据
- 2.降维
- 3.正则化：保留所有的特征，但是减少参数的大小
- 4.集成学习方法：把多个模型集成在一起

成功的机器学习应用不是拥有最好的算法，而是拥有最多的数据！

欠拟合的处理：

- 1.添加新特征
- 2.增加模型复杂度
- 3.减小正则化系数

正则化：

是一种抑制模型复杂度的方法。正则化项一般作为损失函数的一部分被加入到原来的基于数据的损失函数中，基于数据的损失函数称为经验损失，正则化称为结构损失

L1和L2正则化都能使参数尽可能靠近0

代码清单 1-1 加载数据集

```
from sklearn.datasets import load_iris
iris = load_iris()
x = iris.data
y = iris.target
```

评估方法：留出法、交叉验证、自助法

性能度量：均方误差，错误率，精度

实际值	预测值		
		1	0
	1	TP	FN
	0	FP	TN

预测正确与否：T/F

预测结果属性：P/N

1、准确率 (Accuracy)

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2、精准率 (查准率) —— 针对预测结果而言，在预测为正样本的结果中，的确是正样本的概率

$$\text{precision} = \frac{TP}{TP + FP}$$

应用：推荐系统

3、召回率 (查全率，灵敏度) —— 针对实际情况而言，实际为正的样本被预测为正样本的概率

$$\text{recall} = \frac{TP}{TP + FN}$$

应用：医药诊断，假负类要少，查全率要高

4、F1系数：综合查准率和查全率

$$F1 = 2 * P * R / (P + R)$$

5、PR曲线：以精确率为纵坐标，以召回率为横坐标绘制的曲线

通过设定置信度阈值，就可以完成分类。置信度阈值较低时，大量样本被预测为正例，所以召回率较高，而精确率较低；置信度阈值较高时，大量样本被预测为负例，所以召回率较低，而精确率较高。

混淆矩阵

```
from sklearn.metrics import confusion_matrix
```

```
y_true = [1, 0, 1, 1, 0] # 样本实际值
```

```
y_pred = [1, 0, 1, 0, 0] # 模型预测值
```



```
res = confusion_matrix(y_true, y_pred) # 混淆矩阵
```

代码清单 2-1 逻辑回归模型的构造与训练

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.2)
model = LogisticRegression()
model.fit(X_train, y_train)
train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)
print('train score: {train_score:.6f}; test score: {test_score:.6f}'.format(
    train_score=train_score, test_score=test_score))
```

代码清单 2-2 模型评估

```
from sklearn.metrics import recall_score
from sklearn.metrics import precision_score
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
y_pred = model.predict(X_test)
accuracy_score_value = accuracy_score(y_test, y_pred)
recall_score_value = recall_score(y_test, y_pred)
precision_score_value = precision_score(y_test, y_pred)
classification_report_value = classification_report(y_test, y_pred)
print("准确率:", accuracy_score_value)
print("召回率:", recall_score_value)
print("精确率:", precision_score_value)
print(classification_report_value)
```

第2章 线性回归及最大熵模型

回归分析用来建立方程模拟两个或者多个变量之间如何关联

一、线性回归

回归就是用一条曲线对数据点进行拟合，该曲线称为最佳拟合曲线，这个拟合过程称为回归

“线性”是指一次，“回归”实际是拟合，线性回归一般用来做连续值的预测，预测的结果是一个连续值，属于监督学习

线性回归模型：

- (1) 加载数据文件
- (2) 数据预处理
- (3) 建议模型 $f(x) = \theta^T x + b$
- (4) 定义损失函数 $J(\theta)$
- (5) 判断数据和特征值大小，大选择梯度下降法，小选择最小二乘法
- (6) 求解参数，模型校验，经济预测

损失函数： $J(\theta) = 1/2 \sum (y_i - h(x_i))^2$

用最小二乘法求解 $J(\theta) = 1/2 (X\theta - y)^T (X\theta - y)$

令偏导为0，得 $\theta = (X^T X)^{-1} X^T y$

局限：(1) 若逆不存在，无法求解 (2) 样本特征非常多时，不容易求解 (3) 拟合函数非线性不可用

最小二乘法

基于均方误差最小化来进行模型求解的方法称为“最小二乘法”

找到一条直线，使所有样本到直线上的欧式距离之和最小

$$\theta_1 = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} (\sum_{i=1}^m x_i)^2}$$

令目标函数对两个参数的偏导为零可解得：

$$\theta_0 = \frac{1}{m} \sum_{i=1}^m (y_i - \theta_1 x_i)$$

#3 利用解析法 $p = (X^T X)^{-1} X^T Y$

```
pamaters = (((X_mat.T)*X_mat).I)*X_mat.T*Y_mat
```

梯度下降法

批量梯度下降

随即梯度下降

小批量梯度下降

过程：

存在问题：初始值的设置会影响到最终 $J(\theta)$ 的最小值，使得结果可能是局部最小值，也可能是全局最小值。 α 过大，可能会直接越过全局最小值，因此无法找到 $J(\theta)$ 的最小值；若 α 过小，会导致收敛时间过长，但最终会找到全局最优解。目标函数存在多个极小值

二、逻辑回归

```
from sklearn.linear_model import LogisticRegression
```

逻辑回归是一种广义线性回归，其仅在线性回归算法的基础上，套用一个逻辑函数，从而对事件发生的概率进行预测。我们先线性回归中可以得到一个预测值，然后将该值通过逻辑函数进行转换，这样就能够将预测值变成概率值，再根据概率值实现分类。

算法流程：

- (1) 加载数据文件
- (2) 数据预处理，生成多项式特征
- (3) 初始化参数，构建代价函数
- (4) 用梯度下降法优化代价函数，确定参数
- (5) 构建预测函数，求概率值

逻辑回归算法通过拟合一个逻辑函数，即sigmoid函数，将任意的输入映射到 $[0,1]$ 之间， $g(z) = 1/(1+e^{-z})$

- (6) 根据概率值画决策边界

最大熵原理：描述一个概率分布时，在满足所有约束条件的情况下，熵最大的模型时最好的

改进：

逻辑回归主要用于非线性分类问题，具体思路是首先对特征向量进行权重分配之后用sigmoid函数激活。主要有欠拟合和分类和回归精度不高的问题

- 1、欠拟合：原因是特征维度过小，解决思路是增加特征向量维度
- 2、分类和回归精度不高：原因是数据特征有缺失或特征空间很大，解决思路是正则化

逻辑回归分析适用于应变量为分类值的资料

逻辑回归分析属于概率型非线性回归

逻辑回归算法是对事件发生的概率进行预测

2.6 习题

一、填空题

1) 线性回归的目标是求解 ω 和 b ，使得 $f(x)$ 与 y 尽可能接近。求解线性回归模型的基本方法是 最小二乘法。

2) 优化目标是让整个样本集合上的 预测值 与 真实值 之间的欧氏距离之和最小。

3) 多元线性回归问题中： $\omega = (X^T X)^{-1} X^T Y$ ，当 $X^T X$ 可逆 时，线性回归模型存在唯一解。

4) PR 曲线以 精确率 为纵坐标，以 召回率 为横坐标。

5) 在 ROC 曲线与 AUC 曲线中，对于某个二分类分类器来说，输出结果标签（0 还是

1) 往往取决于 置信度 以及 预订的置信度阈值。

二、判断题

1) 逻辑回归是一种广义线性回归，通过回归对数几率的方式将线性回归应用于分类任务。(☒)

2) 信息论中，熵可以度量随机变量的不确定性。现实世界中，不加约束的事物都会朝着“熵增”的方向发展，也就是向不确定性增加的方向发展。(☒)

3) 机器学习中描述一个概率分布时，在满足所有约束条件的情况下，熵最小的模型是最好的。(☐)

4) 准确率可以判断总的正确率，在样本不平衡的情况下，也能作为很好的指标来衡量结果。(☐)

5) 当 $TPR = FPR$ 为一条斜对角线时，表示预测为正样本的结果一半是对的，一半是错的，为随机分类器的预测效果。(☒)

三、单选题

1) 逻辑回归模型解决 (**B**)。

- A. 回归问题 B. 分类问题 C. 聚类问题 D. 推理问题

2) 逻辑回归属于 (**B**) 回归。

- A. 概率性线性 B. 概率性非线性 C. 非概率性线性 D. 非概率性非线性

3) 逻辑回归不能实现 (**D**)。

- A. 二分类 B. 多分类 C. 分类预测 D. 非线性回归

4) 下列关于最大熵模型的表述错误的是 (**B**)。

- A. 最大熵模型是基于熵值越大模型越稳定的假设
B. 最大熵模型使用最大熵原理中一般意义上的熵建模以此缩小模型假设空间
C. 通过定义最大熵模型的参数可以实现与多分类逻辑回归相同的作用
D. 最大熵模型是一种分类算法

5) 下列关于模型评价指标的表述错误的是 (**C**)。

- A. 准确率、精确率、召回率以及 AUC 均是建立在混淆矩阵的基础上
B. 在样本不平衡的条件下准确率并不能作为很好的指标来衡量结果
C. 准确率表示所有被预测为正的样本中实际为正的样本的概率
D. 一般来说，置信度阈值越高，召回率越低，而精确率越高

第3章 K最近邻算法

KNN是一种常用的分类或回归算法，思想：对于分类问题，knn从所有训练样本集合中找到与x最近的k个样本，然后通过投票法选择这k个样本中出现次数最多的类别作为x的预测结果；对于回归问题，knn同样找到与x最近的k个样本，然后对这k个样本的标签求平均，得到x的预测结果。

用于分类任务的机器学习模型称为**分类模型**或**分类器**

相似性度量——欧氏距离

KNN流程：

- 为了判断未知实例的类别，以所有已知类别的实例作为参照选择参数K
- 计算未知实例与所有已知实例的距离
- 选择最近K个已知实例
- 根据少数服从多数的投票法则(majority-voting)，让未知实例归类为K个最邻近样本中最多数的类别

数据标准化：

1、离差标准化

是对原始数据的一种线性变化，结果是对原始数据的数值映射到[0,1]之间，转换公式为 $X = \frac{X - \min}{\max - \min}$

缺点：若数据集中某个数值很大，则离差标准化的值就会接近于0，并且相互之间差别不大。若将来遇到超过目前属性[min,max]取值范围的时候，会引起系统出错，这时便需要重新确定min和max。

2、标准差标准化（零均值标准化，分数标准化）

经过该方法处理的数据均值为0，标准差为1

$$X^* = \frac{X - \bar{X}}{\delta}$$

3、小数定标标准化

通过移动数据的小数位点，将数据映射到区间[-1,1]之间，移动的小数位数取决于数据绝对值的最大值。

$$X^* = \frac{X}{10^k}$$

三种标准化方式优劣：

离差标准化：方法简单，便于理解，标准化后的数据限定在[0,1]区间内

标准差标准化：收数据分布的影响较小

小数定标标准化：适用范围广，并且受到数据分布的影响较小，相比较于前两种方法而言该方法适用程度适中

k折交叉验证

```
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier #导入K近邻分类
scores = cross_val_score(knn, X, Y, cv=5)          #5表示5折交叉验证
```

改进

分组快速搜索近邻法

- 1、将样本集按近邻关系分解成组
- 2、以每组的质心作为代表点和未知样本计算距离
- 3、选出距离最近的一个或若干个组
- 4、在组的范围内应用一般的KNN算法

3.5 习题

一、填空题

- 1) 算法的时间复杂度是 计算到训练集中所有样本的时间加上排序的时间
- 2) 对 k-近邻算法的研究包含三个方面：k值的选取、距离的度量、如何快速的进行k个近邻的检索
- 3) k 值很大时，可以通过交叉验证，在验证集上 多次选取不同的k值 来挑选最佳 k 值。
- 4) 对于连续变量，一般使用 欧式距离 直接进行距离度量。
- 5) 当训练集合的规模很大时，如何快速找到样本 x 的 k个近邻 成为计算机实现近年算法的关键。

二、判断题

1) 投票法的准则是少数服从多数。(☒)

2) 对于离散变量，可以直接使用欧氏距离进行度量。☒ 错错错错错 (对于离散变量, 可以先将离散变量连续化, 然后再使用欧氏距离进行度量)

3) 最近邻算法中, 样本工的预测结果只由训练集中与其距离最近的那个样本决定。

(☒)

4) 对于一般的离散变量同样可以采用类似词嵌入的方法进行距离度量。(☒)

5) 词嵌入是自然语言处理领域常用的一种对单词进行编码的方式。(☒)

三、单选题

1) k-近邻算法的基本要素不包括 (☒)。

A. 距离度量 B. k 值的选择 C. 样本大小 D. 分类决策规则

2) 关于 k-近邻算法说法错误的是 (☒)。

A. k-近邻算法是机器学习 B. k-近邻算法是无监督学习
C. k 代表分类个数 D. k 的选择对分类结果没有影响

3) 以下关于 k-近邻算法的说法中正确的是 (☒)。

A. k-近邻算法不可以用来解决回归问题
B. 随着 k 值的增大, 决策边界会越来越光滑
C. k-近邻算法适合解决高维稀疏数据上的问题
D. 相对 3 近邻模型而言, 1 近邻模型的 bias 更大, variance 更小

4) (☒) 不可以通过无监督学习方式训练。

A. k-近邻算法 B. 决策树 C. RBM D. GAN

5) 以下关于 k-近邻算法的说法中, 错误的是 (☒)。

A. 一般使用投票法进行分类任务
B. k-近邻算法属于懒惰学习
C. k-近邻算法训练时间普遍偏长
D. 距离计算方法不同, 效果也可能有显著差别

第4章 决策树算法

决策树是一种常用的机器学习算法, 既可用于分类, 也可用于回归。思想: 给定一个样本集合, 其中的每个样本由若干属性表示, 决策树通过贪心的策略不断挑选最优的属性。对于离散属性, 以不同的属性值作为节点; 对于连续属性, 以属性值的特定分割点作为节点。决策树拥有很强的数据拟合能力, 往往会出现过拟合现象, 因此需要对决策树进行剪枝, 以减少决策树的复杂度, 提高决策树的泛化能力。

决策树: 从根节点开始一步步走到叶子节点 (决策)

决策树是一种树型结构, 其中每个内部结点表示在一个属性上的测试, 每个分支代表一个测试输出, 每个叶子结点代表一种类别

决策树学习是以实例为基础的归纳学习

决策树学习采用自顶向下的归递方法，基本思想是以信息熵为度量构造一颗熵值下降最快的树，到叶子结点处的熵值为零，此时每个叶子结点的实例都属于同一类

决策树由**决策结点**，**分支**和**叶子结点**组成，通过把数据样本分配到某个叶子结点来确定数据集中样本所属的分类，比较适合分析离散数据，若是连续则转换为离散

算法（不同的目标函数）：

- 1、基于**信息增益**的ID3算法
- 2、基于**信息增益率**的C4.5算法
- 3、基于**基尼指数**的CART算法

熵：表示随即变量不确定性的度量（物体内部混乱程度）

$H(X) = -\sum p_i \times \log p_i$ （有负号）

信息熵计算题目

假如有一个普通骰子A，扔出1-6的概率都是1/6

有一个骰子B，扔出6的概率是50%，扔出1-5的概率都是10%

有一个骰子C，扔出6的概率是100%。

$$\text{骰子A} : -\left(\frac{1}{6} \times \log_2 \frac{1}{6}\right) \times 6 \approx 2.585$$

$$\text{骰子B} : -\left(\frac{1}{10} \times \log_2 \frac{1}{10}\right) \times 5 - \frac{1}{2} \times \log_2 \frac{1}{2} \approx 2.161$$

$$\text{骰子C} : -(1 \times \log_2 1) = 0$$

算法流程：

- 1、加载数据文件
- 2、计算数据集s每个属性的熵
- 3、选取数据集s中熵最小的属性
- 4、从s中剔除该属性
- 5、添加该属性作为决策树的结点
- 6、重复3-5直达s为空

1、ID3算法

每次选择信息增益最大的特征作为结点，**信息增益**即决策树上某个分支上整个数据集信息熵和当前结点信息熵差值

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

$$Gain(A) = Info(D) - Info_A(D)$$

选择结点(信息增益max)计算题目

信息增益(Information Gain): $Gain(A) = Info(D) - Info_A(D)$

R/D	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

$$Info(D) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

$$Info_{age}(D) = \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5}\right) + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4}\right) + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}\right) = 0.694 \text{ bits.}$$

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

类似:

$$Gain(income) = 0.029,$$

$$Gain(student) = 0.151,$$

$$Gain(credit_rating) = 0.048$$

ps: 计算info age(D)时, 概率是age单一特征占比全部, info (Di) 是计算在这个单一特征中, 对应的yes和no的概率, 几个取值算几个熵求和

缺点: 1、不能处理连续分布的数据特征

2、ID3算法往往选择取值较多的分支属性的问题

2、C4.5算法

$$SplitInfo_A(D) = - \sum_{i=1}^v \frac{|D_i|}{|D|} \times \log_2 \left(\frac{|D_i|}{|D|} \right)$$

$$GrianRate(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

$$Gain_ratio(outlook) = \frac{Gain(outlook)}{- \sum_{i=1}^3 \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}} = \frac{0.247}{-(\frac{5}{14} \log_2 \frac{5}{14} + \frac{5}{14} \log_2 \frac{5}{14} + \frac{4}{14} \log_2 \frac{4}{14})}$$

C4.5算法使用信息增益率代替信息增益，进行特征选择，克服了信息增益选择特征时偏向于特征值个数较多的不足；

C4.5能够完成对连续属性的离散化处理，能够对不完整数据进行处理

3、C5.0算法

对C4.5改进，目的是对含有大量数据的数据集进行分析

优势：构建时间短，生成的决策树规模要小，拥有更少的叶子结点数；使用了提升法，组合多个决策树作出分类。

4、CART算法

递归的构建二叉决策树的过程，使用基尼（Gini）系数最小化准则来进行特征选择
样本集合的基尼值越低，集合的纯度越高。CART生成的是二叉树

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2$$
$$Gini_A(D) = \frac{|D_1|}{|D|}Gini(D_1) + \frac{|D_2|}{|D|}Gini(D_2)$$
$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

天气与打球关联关系表					
Day	Outlook	Temp	Humidity	Windy	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

$Gini(windy) = \frac{|S_1|}{|S|}Gini(S_1) + \frac{|S_2|}{|S|}Gini(S_2) = \frac{8}{14} \times \left[1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2 \right] + \frac{6}{14} \times \left[1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 \right]$

CART算法的主要流程：

- 1、输入需要分类的数据集和类别标签。
- 2、使用最小剩余方差判定回归树的最优划分，并创建特征的划分结点。最小剩余方差子函数，计算数据集各列的最优划分方差、划分列、划分值。

- 3、在划分结点，使用二分数据集子函数将数据集划分为两部分。根据二分数据的结果，构建出新的左、右结点，作为树生长出的两个分支。
 - 4、检验是否符合递归的终止条件。
 - 5、将划分的新结点包含的数据集和类别标签作为输入，递归执行上述步骤。
- sklearn.tree提供了CART算法的实现方法：DecisionTreeRegressor

算法关键问题：

1、递归结束条件

直到：叶子结点纯了---仅包含一类实例

达到最大深度

达到某一性能指标

2、连续属性离散化

分类数据有二元属性、标称属性

二元属性：只有两个可能值

标称属性：存在多个可能值，存在两种方式：多路划分和二元划分

-对于ID3、C4.5等算法，均采用多分支划分的方法，标称属性有多少种可能的取值，就设计多少个分支

-CART算法采用二分递归分割的方法，因此该算法生成的决策树均为二叉树

3、过拟合问题

模型的训练误差比较高，则称此分类模型欠拟合

模型的训练误差低但是泛化误差比较高，则称此分类模型过拟合

对于欠拟合问题，可以通过**增加分类属性的数量、选取合适的分类属性等方法**，提高模型对于训练样本的拟合程度

决策树剪枝

剪枝是用来缩小决策树的规模，从而降低最终算法的复杂度并提高预测准确度，包括**预剪枝**和**后剪枝**两类

(1) 预剪枝的思路是提前终止决策树的生长，在形成完全拟合训练样本集的决策树之前就停止树的生长，避免决策树规模过大而产生过拟合

(2) 后剪枝的思路是**先让决策树完全生长**，之后针对子树进行判断，**用叶子结点或者子树中最常用的分支替换子树**，以此方式不断改进决策树，直到无法改进为止

错误率降低剪枝（REP）是**后剪枝**策略中最简单的算法之一

代价复杂度剪枝策略(CCP)

一般情形下，**后剪枝**决策树的欠拟合风险很小，泛化性能往往优于预剪枝决策树。但后剪枝训练时间比未剪枝和预剪枝决策树都要大得多。

4.3.1 预剪枝

对决策树进行预剪枝时一般通过验证集进行辅助。每次选择信息增益最大的属性进行划分时，首先在验证集上对模型进行测试，如果划分之后能够提高验证集的准确率，则进行划分；否则，将当前节点作为叶节点，并以当前节点包含的样本中出现次数最多的样本作为当前节点的预测值。

由于决策树本身是一种贪心的策略，并不一定能够得到全局的最优解。使用预剪枝的策略容易造成决策树的欠拟合。

4.3.2 后剪枝

对于一棵树，其代价函数定义为经验损失和结构损失两个部分：经验损失是对模型性能的度量，结构损失是指对模型复杂度的度量。根据奥卡姆剃刀原则，决策树模型性能应尽可能地性能高，复杂度应尽可能地低。经验损失可以使用每个叶节点上的样本分布的熵之和来描述，结构损失可以用叶节点的个数来描述。设决策树 T 中叶节点的数目为 M ，代价函数的

代码使用

```
from sklearn import tree
clf=tree.DecisionTreeClassifier (criterion="gini/entropy") #初始化
Clf=clf.fit (X,Y) #训练决策树
res=Clf.predict () #预测
from sklearn.model_selection import cross_val_score
cross_val_score(clf,iris.data,iris.target,cv=10) #10折交叉验证

#把数据转换成01表示
vec = DictVectorizer()
x_data = vec.fit_transform(featureList).toarray()
lb = preprocessing.LabelBinarizer()
y_data = lb.fit_transform(labelList)
```

2) 在构建决策树时，需要计算每个用来划分数据特征的得分，选择分数最高的特征。以下可以作为得分的是 (**D 不太懂**)

A. 熵

B. 基尼系数

C. 训练误差

D. 以上都是

4.5 习题

一、填空题

- 1) 决策树是一种常用的机器学习算法，既可用于 分类，也可用于 回归。
- 2) 决策树拥有很强的数据拟合能力，往往会产生 过拟合 现象，因此需要对决策树进行剪枝。
- 3) 信息增益 是最早用于决策树模型的特征选择指标，也是 ID3 算法的核心。
- 4) 信息增益比定义为 信息增益与数据集在属性 A_i 上的分布的熵之比。
- 5) 使用预剪枝的策略容易造成决策树的 欠拟合。

二、判断题

- 1) 对决策树进行剪枝可以减小决策树的复杂度，提高决策树的专一性能力。 (☒ 错)
- 2) 经验损失可以使用每个子树上的样本分布的熵之和来描述。 (☒ 错)
- 3) 结构损失可以用叶节点的个数来描述。 (☒ 对)
- 4) 决策树本身是一种贪心的策略，不一定能够得到全局的最优解。 (☒ 对)
- 5) 由于 train_test_split 函数在划分数据集时存在一定的随机性，所以重复运行上述代码可能会得到不同的准确率。 (☒ 对)

三、单选题

- 1) 关于机器学习中的决策树学习，说法错误的是 (A)。
A. 受生物进化启发 B. 属于归纳推理
C. 用于分类和预测 D. 自顶向下递推
- 2) 在构建决策树时，需要计算每个用来划分数据特征的得分，选择分数最高的特征，以下可以作为得分的是 (D)。
A. 熵 B. 基尼系数 C. 训练误差 D. 以上都是
- 3) 在决策树学习过程中，(D) 可能会导致问题数据（特征相同但是标签不同）。
A. 数据噪音 B. 现有特征不足以区分或决策
C. 数据错误 D. 以上都是
- 4) 根据信息增益来构造决策树的算法是 (A)。
A. ID3 决策树 B. 递归 C. 归约 D. FIFO
- 5) 决策树构成顺序是 (A)。
A. 特征选择、决策树生成、决策树剪枝
B. 决策树剪枝、特征选择、决策树生成
C. 决策树生成、决策树剪枝、特征选择
D. 特征选择、决策树剪枝、决策树生成

第5章 朴素贝叶斯算法

朴素贝叶斯分类器是一种有监督的统计学过滤器，常用于垃圾邮件过滤，信息检索等领域

先验信息：抽样之前，有关推断问题中未知参数的一些信息，通常来自经验或历史资料

朴素贝叶斯法是基于贝叶斯定理与特征属性独立假设的**分类**方法，属于**监督学习**的生成模型。

朴素贝叶斯是典型的生成学习方法，适合小规模数据，适合进行多分类任务

贝叶斯定理： $P(A | B) = P(B|A)P(A)/P(B)$ 提供了从先验概率 $p(A)$ 计算后验概率 $p(A | B)$ 的方法

朴素贝叶斯分类器可以看作：对于给定的待分类项，求解在该分类项出现的条件下，各个类别出现的概率。根据各个类别出现的概率，哪个最大，就将其划归为那个类别。

$$P(\text{yes}|\text{sunny, cool, high, strong}) = \frac{P(\text{sunny}|\text{yes}) * P(\text{cool}|\text{yes}) * P(\text{high}|\text{yes}) * P(\text{strong}|\text{yes}) * P(\text{yes})}{P(\text{high}|\text{yes}) * P(\text{strong}|\text{yes}) * P(\text{yes})}$$

$$P(\text{no}|\text{sunny, cool, high, strong}) = \frac{P(\text{sunny}|\text{no}) * P(\text{cool}|\text{no}) * P(\text{high}|\text{no}) * P(\text{strong}|\text{no}) * P(\text{no})}{P(\text{high}|\text{no}) * P(\text{strong}|\text{no}) * P(\text{no})}$$

拉普拉斯平滑技术：在有些类别里没有的特征将会导致这些条件概率值为‘0’，解决方案是在这这些条件概率的分子和分母各加一

朴素贝叶斯有稳定的分类效率，在大量样本下会有较好的表现，对小规模的数据仍然有效，能处理多分类任务。朴素贝叶斯适合增量式训练，即可以实时对新增样本进行训练

算法流程：

- 1、确定特征属性，获取训练样本集合
- 2、计算各类别的先验概率 $P(Y=C_k)$
- 3、计算各类别下的各特征属性的条件概率 $P(X_j=x_j | Y=C_k)$
- 4、计算各类别的后验概率 $P(Y=C_k | X=x)$
- 5、后验概率最大项作为样本所属类别

代码：

生成的50000个随机样本

```
X, y=make_blobs
```

```
(n_samples=50000,n_feature=2,cluster_std=1.0,center=,shuffle=False,random_state=42)
```

```
clf=GaussianNB()
```

sklearn中的朴素贝叶斯

`naive_bayes.GaussianNB` (priors=None先验概率) 高斯朴素贝叶斯

`naive_bayes.MultinomialNB` 针对多项式模型的朴素贝叶斯分类器

naive_bayes.BernoulliNB 针对多元伯努利模型的朴素 贝叶斯分类器

from sklearn.naive_bayes import MultinomialNB as NB

习题：

5.6 习题

一、填空题

- 1) 朴素贝叶斯分类器 是一种有监督的统计学过滤器，在垃圾邮件过滤、信息检索等领域十分常用。
- 2) 极大似然估计的思想是：找到这样一个参数 p ，它使所有随机变量的 联合概率 最大。
- 3) 极大似然估计通过求解参数，使得 $f(x_1, x_2, x_3, \dots, x_N; \theta)$ 最大，这是一种很朴素的思想。
- 4) 朴素贝叶斯分类器假设样本特征之间 相互独立。
- 5) 当样本集合不够大时，可能无法覆盖特征的所有可能取值，为了避免这样的问题，实际应用中常采用 平滑处理（最常用的是拉普拉斯平滑）。

二、判断题

- 1) 朴素贝叶斯思想的本质是极大似然估计。(☒)
- 2) 用极大似然法估计值时, 概率分布模型只能是连续的。(☒)
连续的是概率密度函数, 离散的是概率质量函数
- 3) 已知整体的概率分布模型 $f(x; \theta)$, 但是模型的参数 θ 未知时, 可以用极大似然估计来估计 θ 的值。(☒)
- 4) 在运用极大似然法求解概率时, 令似然函数 $L(\theta)$ 对参数 θ 的导数为 0, 可以得到 θ 的最优解。(☒)
- 5) 概率论贝叶斯公式描述中, $P(Y_i)$ 称为先验概率, $P(Y_i|X)$ 称为后验概率。(☒)

三、单选题

- 1) 朴素贝叶斯分类器的特征不包括 (☒ D 错, C 对)
A. 孤立的噪声点对该分类器影响不大
B. 数据的缺失值影响不大
C. 要求数据的属性相互独立
D. 条件独立的假设可能不成立
- 2) 朴素贝叶斯分类器基于 (☒ A 错 B 对) 假设。
A. 样本分布独立性 B. 属性条件独立性 C. 后验概率已知 D. 先验概率已知
- 3) 下列关于朴素贝叶斯分类器错误的是 (☒ D)。
A. 朴素贝叶斯模型发源于古典数学理论, 有稳定的分类效率
B. 对小规模的数据表现很好, 能处理多分类任务, 适合增量式训练
C. 对缺失数据不太敏感, 算法也比较简单, 常用于文本分类
D. 对输入数据的表达形式不敏感
- 4) 朴素贝叶斯分类器为 (☒ A)。
A. 生成模型 B. 判别模型 C. 统计模型 D. 预算模型
- 5) 下列关于朴素贝叶斯分类器正确的是 (☒ D)。
A. 朴素贝叶斯分类器的变量必须是非连续型变量

- B. 朴素贝叶斯模型中的特征和类别变量之间也要相互独立
- C. 朴素贝叶斯分类器对于小样本数据集效果不如决策树好
- D. 朴素贝叶斯模型分类时需要计算各种类别的概率, 取其中概率最大者为分类预测值

四、简答题

- 1) 极大似然估计和贝叶斯估计在思想上有何区别?
- 2) 拉普拉斯平滑解决了什么问题? 请举例说明。
- 3) 朴素贝叶斯分类器为什么是“朴素”的?

1、贝叶斯决策中关键便在于知道后验概率, 那么问题便集中在求解类条件概率密度! 那么如何求呢?
答案便是: 将类条件概率密度进行参数化。最大似然估计和贝叶斯估计都属于参数化估计。
二者的最大区别那就是: 参数

最大似然估计要估计的参数 θ 被当作是固定形式的一个未知变量，然后我们结合真实数据通过最大化似然函数来求解这个固定形式的未知变量。

贝叶斯估计则是将参数视为是有某种已知先验分布的随机变量，意思便是这个参数他不是一个固定的未知数，而是符合一定先验分布如：随机变量 θ 符合正态分布等！那么在贝叶斯估计中除了类条件概率密度 $p(x|w)$ 符合一定的先验分布，参数 θ 也符合一定的先验分布。我们通过贝叶斯规则将参数的先验分布转化成后验分布进行求解。

贝叶斯估计用的是后验概率，而最大似然估计直接使用是类条件概率密度。

2、拉普拉斯平滑是为了解决零概率的问题。

零概率问题：在计算事件的概率时，如果某个事件在观察样本库（训练集）中没有出现过，会导致该事假的概率结果是0.这是不合理的。分子加一，分母加K，K代表类别数目。

举例：

假设在文本分类中，有3个类：C1、C2、C3。

在指定的训练样本中，某个词语K1，在各个类中观测计数分别为0，990，10。

则对应K1的概率为0，0.99，0.01。

显然C1类中概率为0，不符合实际。

于是对这三个量使用拉普拉斯平滑的计算方法如下：

$$1/1003 = 0.001, 991/1003=0.988, 11/1003=0.011$$

别的类也要加，分母加上类别总数。

3、之所以被称为“朴素”，是因为它假定所有的特征在数据集中的作用是同样重要和独立的，正如我们所知，这个假设在现实世界中是很不真实的，因此，说是很“朴素的”。

第6章 支持向量机

支持向量机是一种基于统计学习理论的监督学习方法

- 线性可分支持向量机
 - 硬间隔最大化
 - 硬间隔支持向量机
- 线性支持向量机
 - 硬间隔最大化
 - 硬间隔支持向量机
- 非线性支持向量机
 - 核函数

当训练数据**线性可分**时，通过硬间隔最大化，学习一个线性可分支持向量机；

当训练数据**近似线性可分**时，通过软间隔最大化，学习一个线性支持向量机；

当训练数据**不可分**时，通过使用核技巧以及软间隔最大化，学习一个非线性支持向量机

！重要！！！！算法流程

线性支持向量机的算法步骤如下：

- (1) 给定训练集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}, y = \{-1, 1\}$;
- (2) 构造最优化问题：

$$\min_{\alpha} \varphi(\alpha) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i$$

- (3) 计算参数 w 和 b ：

$$w = \sum_{i=1}^N \alpha_i y_i x_i$$
$$b = y_i - x_i \cdot \sum_{i=1}^N \alpha_i y_i x_i$$

- (4) 得出超平面与决策函数：

$$w \cdot x + b = 0$$

分类决策函数如下所示：

$$f(x) = \text{sign}(w \cdot x + b)$$

$y_i = -1/+1$

称样本数据集为线性可分；称 $w^T X + b = 0$ 为分离超平面

对于一个线性可分性的样本数据集，其分离超平面通常不止一个。线性可分支持向量机提出了**间隔最大化条件**，以此来求得距离两个可分类的**最佳超平面**。

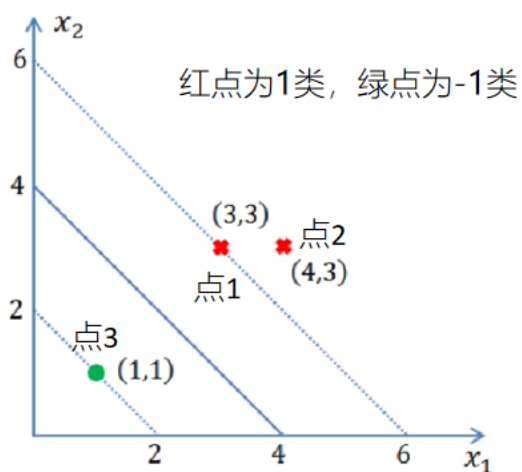
SVM寻找区分两类的超平面（hyper plane），使边缘(margin)最大

样本数据点 X 到该超平面的距离表示为： $|w^T X + b|$

如何判断是否分类正确？

$w^T X + b$ 与标记 y 符号是否一致。若一致，则分类正确；否则，分类错误。可用 $y(w^T X + b)$ 替代 $|w^T X + b|$ 表示样本数据点到超平面的距离。

例子：



可知目标函数为

$$\min_{\alpha} f(\alpha), \quad \text{s.t. } \alpha_1 + \alpha_2 - \alpha_3 = 0, \quad \alpha_i \geq 0, i = 1, 2, 3$$

其中

$$f(\alpha) = \frac{1}{2} \sum_{i,j=1}^3 \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^3 \alpha_i$$

求内积

$$= \frac{1}{2} (18\alpha_1^2 + 25\alpha_2^2 + 2\alpha_3^2 + 42\alpha_1\alpha_2 - 12\alpha_1\alpha_3 - 14\alpha_2\alpha_3) - \alpha_1 - \alpha_2 - \alpha_3$$

然后，将 $\alpha_3 = \alpha_1 + \alpha_2$ 带入目标函数，得到一个关于 α_1 和 α_2 的函数

$$s(\alpha_1, \alpha_2) = 4\alpha_1^2 + \frac{13}{2}\alpha_2^2 + 10\alpha_1\alpha_2 - 2\alpha_1 - 2\alpha_2$$

对 α_1 和 α_2 求偏导数并令其为0，易知 $s(\alpha_1, \alpha_2)$ 在点 $(1.5, -1)$ 处取极值。而该点不满足 $\alpha_i \geq 0$ 的约束条件，于是可以推断最小值在边界上达到。经计算当 $\alpha_1 = 0$ 时， $s(\alpha_1 = 0, \alpha_2 = 2/13) = -0.1538$ ；当 $\alpha_2 = 0$ 时， $s(\alpha_1 = 1/4, \alpha_2 = 0) = -0.25$ 。于是 $s(\alpha_1, \alpha_2)$ 在 $\alpha_1 = 1/4, \alpha_2 = 0$ 时取得最小值，此时亦可算出 $\alpha_3 = \alpha_1 + \alpha_2 = 1/4$ 。因为 α_1 和 α_3 不等于0，所以对应的点 x_1 和 x_3 就应该是支持向量。

进而可以求得

$$\mathbf{w}^* = \sum_{i=1}^3 \alpha_i^* y_i x_i = \frac{1}{4} \times (3, 3) - \frac{1}{4} \times (1, 1) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

即 $w_1 = w_2 = 0.5$ 。进而有

$$b^* = 1 - (w_1, w_2) \cdot (3, 3) = -2$$

因此最大间隔分类超平面为

$$\frac{1}{2}x_1 + \frac{1}{2}x_2 - 2 = 0$$

分类决策函数为

$$f(x) = \text{sign}\left(\frac{1}{2}x_1 + \frac{1}{2}x_2 - 2\right)$$

支持向量：真正发挥作用的数据点， α 不为0的点

硬间隔分类的缺点：1、只对数据线性分离才有效2、对异常值太敏感

svm模型：模型对S中任何训练样本都不能作出错误分类

软间隔SVM模型：允许对少量训练样本出现分类错误，引入一个松弛变量，将约束条件转化为：

$$y_i(\mathbf{w}^T X_i + b) \geq 1 - \xi_i$$

ξ_i 的取值越大，模型对错误分类的容忍程度越高。

可得如下优化求解软间隔SVM模型的目标函数：

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i ; \text{ s.t. } y_i(\mathbf{w}^T X_i + b) \geq 1 - \xi_i$$

其中 $C > 0$ 是为惩罚因子。

非线性分类问题：支持向量机选择一个核函数，通过将数据映射到高维空间，从而解决原始空间中线性不可分的问题。

核方法的核心思想是，利用核函数直接计算映射到空间后实例间的内积，以此代替先做映射再做内积。

常用的核函数有

- 1) 线性核函数 (Linear Kernel)，主要用于线性可分的情况
- 2) 多项式核函数 (Polynomial Kernel)，一种非稳态核函数，适合于正交归一化后的数据
- 3) 径向基核函数 (Radial Basis Function Kernel) 又称高斯核
- 4) Sigmoid核函数 (Sigmoid Kernel)

代码：支持向量机是一种可构造唯一决策面的线性分类器

```
class sklearn.svm.LinearSVC (c=10)
```

C: 惩罚系数, 即对误差的宽容度。C越大, 说明越不能容忍出现误差, 容易过拟合; C越小, 越容易欠拟合。需要对C进行选择, 以便在平滑的决策边界和分类正确性之间进行平衡。

make_blobs: 多类单标签数据集, 为每个类分配一个或多个正态分布的点集

```
from sklearn.datasets.samples_generator import make_blobs
```

基本语法: `sklearn.datasets.samples_generator.make_blobs(n_samples=100点的总数, n_features=2特征维度, centers=3产生中心点的数量, cluster_std=1.0标准差, center_box=(-10.0, 10.0), shuffle=True, random_state=None)`

返回结果: X: [n_samples,n_features]大小的特征矩阵 y: [n_samples]大小的标签数据矩阵, 对应特征矩阵的每一行

项目中选用的模型是 `sklearn` 提供的 SVC, 其构造函数的 `kernel` 参数可以选择, 具体如下:

- 1) `linear`: 线性核函数。
- 2) `poly`: 多项式核函数。
- 3) `rbf`: 径向基核函数/高斯核。
- 4) `sigmod`: `sigmod` 核函数。
- 5) `precomputed`: 提前计算好核函数矩阵。

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.svm import LinearSVC
```

```
from sklearn.datasets import make_blobs
```

创建随机分布的, 可以分离的400个样本点

```
X, y = make_blobs(n_samples=400, centers=2, random_state=32)
```

创建LinearSVC对象

```
clf = LinearSVC(C=10)
```

```
clf.fit(X, y)
```

```
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.Paired)
```

```
print (clf.support_vectors_) #得到支持向量
```

```
print (clf.support_) #得到支持向量的索引
```

```
print (clf.n_support_) #得到每一类的支持向量的个数
```

画出决策函数

```
ax = plt.gca()
```

```
xlim = ax.get_xlim()
```

```
ylim = ax.get_ylim()
```

网格化评价模型


```

xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = clf.decision_function(xy).reshape(XX.shape)
画分类边界
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
linestyles=['--', '-', '--'])
plt.title("Maximum margin using LinearSVC")
plt.show()

```

第七章 集成学习

集成方法是用多种学习方法的组合来获取比原方法更优的结果。

集成学习主要包括3个部分：个体的生成方法、基学习器（个体学习器）和结论的合（集）成方法。同质个体学习器按照个体学习器之间是否存在依赖关系又可以分为两类：存在着强依赖关系（串行Boosting算法），不存在强依赖关系（并行Bagging和随机森林（Random Forest）算法）区别在于：集成方式是并行还是串行

随机森林

随机森林以决策树为基分类器，最终结果由单个决策树的输出结果投票决定，从原始的样本数据集中使用Bootstrap方法随机抽取（有放回）样本，也称“自助法”，cart回归树：平方误差最小化；cart分类树：gini

训练过程中：样本扰动且属性扰动

随机森林和决策树可以不做归一化处理

sklearn.ensemble.RandomForestClassifier (n_estimators=10树的数量,criterion='gini'默认, entropy表示信息熵)

对随机森林分类器和决策树分类器进行比较：

导入内置数据集模块

```
from sklearn.datasets import load_breast_cancer
```

导入sklearn模块中的决策树分类器类和随机森林分类器类

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

导入sklearn模块中的模型验证类

```

from sklearn.model_selection import train_test_split, cross_val_score
import matplotlib.pyplot as plt
导入乳腺癌数据集
cancer = load_breast_cancer()
使用train_test_split函数自动分割训练集与测试集，其中test_size为测试集所占比例
x_train, x_test, y_train, y_test = train_test_split(cancer.data, cancer.target, test_size=0.3)
定义一个决策树分类器对象用于做比较
dt = DecisionTreeClassifier(random_state=0)
定义一个随机森林分类器对象
rf = RandomForestClassifier(random_state=0)
dt.fit(x_train, y_train)
rf.fit(x_train, y_train)
score_dt = dt.score(x_test, y_test)
score_rf = rf.score(x_test, y_test)
输出准确率
print('Single Tree : ', score_dt)
print('Random Forest : ', score_rf)
dt_scores = []
rf_scores = []
使用cross_val_score进行交叉验证，其中：
cv为份数，即将数据集划分为n分，依次取每一份做测试集，其他n-1份做训练集，
返回每次测试准确率评分的列表
for i in range(10):
    rf_score = cross_val_score(RandomForestClassifier(n_estimators=25), cancer.data,
                                cancer.target, cv=10).mean()
    rf_scores.append(rf_score)
    dt_score = cross_val_score(DecisionTreeClassifier(), cancer.data, cancer.target, cv=10).mean()
    dt_scores.append(dt_score)
观察弱分类器数量对分类准确度的影响
rf_scores = []
for i in range(1, 50):
    rf = RandomForestClassifier(n_estimators=i)
    rf_score = cross_val_score(rf, cancer.data, cancer.target, cv=10).mean()
    rf_scores.append(rf_score)
plt.figure()
plt.title('Random Forest')

```

```
plt.xlabel('n_estimators')
plt.ylabel('Accuracy')
plt.plot(range(1,50),rf_scores)
plt.show()
```

提升法

Boosting是一种可将弱学习器提升为强学习器的算法，分为训练和检验两个阶段

AdaBoost算法的工作机制

首先，训练集用初始权重训练出一个弱学习器1，根据弱学习的学习误差率表现来更新训练样本的权重，使得之前弱学习器1学习误差率高的训练样本点的权重变高，使得这些误差率高的点在后面的弱学习器2中得到更多的重视。

然后，基于调整权重后的训练集来训练弱学习器2，如此重复进行，直到训练到指定的弱学习器数量。

最后，将这些弱学习器通过集合策略进行整合，得到最终的强学习器。

```
sklearn.ensemble.AdaBoostClassifier
```

梯度提升法GBDT

种用于回归和分类问题的集成学习方法，生成一个由弱学习器（通常是决策树）组成的强学习器。提升决策树的弱学习器只使用CART回归树模型

```
sklearn.ensemble.GradientBoostingClassifier(learning_rate=0.1,n_estimators=100,subsample=1.0)
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

```
gbc = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)
```

#测试learning_rate参数对分类效果的影响

```
gbc_scores = []
```

```
for i in np.arange(0.1,1,0.05):
```

```
gbc.learning_rate = i
```

```
gbc.fit(x_train,y_train)
```

```
gbc_score = gbc.score(x_test,y_test)
```

```
gbc_scores.append(gbc_score)
```

#使用cross_val_score进行交叉验证

```
for i in range(20):
```

```
gbc_score = cross_val_score(gbc, cancer.data, cancer.target,cv=10).mean()
```

```
gbc_scores.append(gbc_score)
```

```
dt_score = cross_val_score(dt, cancer.data, cancer.target,cv=10).mean()
```

```
dt_scores.append(dt_score)
```

偏差和方差

1、偏差是指预测结果与真实值之间的差异，更多的是针对某个模型输出的样本误差

2、方差是指多个模型输出结果之间的离散差异，不是针对某一个模型输出样本进行判定

泛化误差可以分解为偏差（Biase）、方差（Variance）和噪声（Noise）。

Bias是用所有可能的训练数据集训练出的所有模型的输出的平均值与真实模型的输出值之间的差异。

Variance是不同的训练数据集训练出的模型输出值之间的差异。

Bias的对象是单个模型，是期望输出与真实标记的差别。它描述了模型对本训练集的拟合程度。

Variance的对象是多个模型，是相同分布的不同数据集训练出模型的输出值之间的差异。它刻画的是数据扰动对模型的影响。

欠拟合：模型不能适配训练样本，有一个很大的偏差

过拟合：模型很好的适配训练样本，但在测试集上表现很糟，有一个很大的方差

随着模型复杂度的增加，偏差越来越小，方差越来越大

Bagging优化的对象是模型的方差，对偏差影响小

第9章 PCA降维

机器学习领域中的降维就是指采用**某种映射方法**，将原高维空间中的数据点映射到低维度的空间中。通过降维，可以减少冗余信息所造成的误差，提高识别的精度。此外，通过降维可以寻找数据内部的本质结构特征。

降维对数据的处理主要包含特征筛选和特征提取

降维方法可分为线性降维和非线性降维两大类。

1、**线性降维**假设构成数据集的各变量间**独立无关**。

- 主成分分析（Principal Component Analysis, PCA）
- 独立成分分析（Independent Component Analysis, ICA）
- 线性判别分析（Linear Discriminant Analysis, LDA）

2、**非线性降维**方法，也称之为流形学习方法（Manifold Learning），其目标是从高维采样数据中恢复低维流形结构，**不改变数据本身的拓扑特性**。

- 基于保持全局结构信息，如等距离映射算法ISOMAP
- 关注局部结构信息，如LLE、LE、HLLE

主成分分析PCA

PCA可以把具体相关性的高维变量合成线性无关的低维变量，称为**主成分**。主成分能够尽可能保留原始数据的信息。并期望在所投影的维度上数据的方差最大

是一种掌握事物**主要矛盾**的统计分析方法，它可以从多元事物中解析出主要影响因素，揭示事物的本质，简化复杂的问题。

找到数据最重要的方向：方差最大的方向

主成分分析使用样本集合中对应子变量上取值的方差来表示该特征的重要程度，方差越大，特征的重要程度越大；方差越小，特征的重要程度越小。

假设变量之间无关

算法流程：

- (1) 将原始数据按行组成m行n列的矩阵X；
- (2) 将X的**每一列**（代表一个属性字段）进行零均值化，即减去这一列的均值；
- (3) 求出协方差矩阵；
- (4) 求出协方差矩阵的特征值及对应的特征向量r；
- (5) 将特征向量按对应特征值大小从左到右按列排列成矩阵，取前k列组成矩阵P；（主成分矩阵）
- (6) 计算降维到k维的数据

方差描述一个数据的离散程度

协方差描述两个数据的相关性【-1,1】负相关到正相关，只能处理二维问题

代码：

```
def zeroMean(dataMat):
    #按列求平均值，即求各个特征的均值
    meanVal = np.mean(dataMat,axis = 0)
    #计算该轴上的统计值（0为列，1为行）
    newData = dataMat - meanVal
    return newData,meanVal

def pca(dataMat,percent=0.19):
    #求协方差矩阵，numpy中的cov函数用于求协方差矩阵，参数rowvar很重要，#若rowvar=0，说明
    #传入的数据一行代表一个样本，若非0，说明传入的数据一列代表一个样本。
    #因为newData每一行代表一个样本，所以将rowval设置为0 covMat即所求的协方差矩阵
    newData,meanVal=zeroMean(dataMat)
    covMat=np.cov(newData,rowvar=0)

    eigValIndice=np.argsort(eigVals)    #对特征值从小到大排序
    n_eigValIndice=eigValIndice[-1:-(n+1):-1] #最大的n个特征值的下标
    n_eigVect=eigVects[:,n_eigValIndice]    #最大的n个特征值对应的特征向量
    lowDDataMat=newData * n_eigVect        #低维特征空间的数据
    reconMat=(lowDDataMat * n_eigVect.T) + meanVal #重构数据
    return reconMat,lowDDataMat,n
```



```
sklearn.decomposition.PCA(n_components主成分的个数, svd_solver特征值分析的方法)
pca=PCA (n_components=2)
reduced_X=pca.fit_transform(X)
```

代码清单 9-1 鸢尾花数据集加载与归一化

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import scale
iris = load_iris()
data, targets = scale(iris.data), iris.target
```

代码清单 9-2 PCA 降维鸢尾花数据集

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
y = pca.fit_transform(data)
```

优缺点

PCA算法的优点：

- (1) 仅仅需要以方差衡量信息量，不受数据集以外的因素影响。
- (2) 各主成分之间正交，可消除原始数据成分间的相互影响的因素。
- (3) 计算方法简单，主要运算是特征值分解，易于实现。

PCA算法的缺点：

- (1) 主成分各个特征维度的含义具有一定的模糊性，不如原始样本特征的解释性强。
- (2) 方差小的非主成分也可能含有对样本差异的重要信息，降维丢弃的数据可能对后续数据处理有影响。

优化和改进

- 1、KPCA算法：**一种改进的PCA非线性降维算法，它利用核函数的思想，把样本数据进行非线性变换，然后在变换空间进行PCA，这样就实现了非线性PCA。（PCA是一种线性特征提取算法）
- 2、局部PCA算法：**局部PCA是一种改进的PCA局部降维算法，它在寻找主成分时加入一项具有局部光滑性的正则项，从而使主成分保留更多的局部性信息。

第10章 聚类算法

无监督学习

无监督学习最常应用的场景是聚类(clustering)和降维(Dimension Reduction)

聚类就是根据数据的“相似性”将数据分为多类的过程

分类：学习 / 训练过程有监督，训练样本有明确标签

聚类：学习 / 训练过程无监督，样本无明确标签

聚类生成的子集合称为**簇**，生成的**簇内部**的任意两个对象之间具有**较高的相似度**；属于**不同簇**的两个对象间具有**较高的相异度**

聚类通常作为其他数据挖掘或建模的前奏

基于距离的分类

1、闵可夫斯基距离

$$\text{dist}(\mathbf{X}, \mathbf{Y}) = \sqrt[p]{\sum_{k=1}^n (|x_k - y_k|^p)}$$

该距离最常用的p是2和1，前者是欧几里得距离，后者是曼哈顿距离。当p趋近于无穷大时，闵可夫斯基距离转化成切比雪夫距离。

2、欧式距离

$$\text{dist}(\mathbf{X}, \mathbf{Y}) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2}$$

```
import numpy as np
```

```
import scipy.spatial.distance as dis
```

```
X = np.vstack([x,y]) # 向量x, y在前面例子中已经赋值。下同
```

```
dist = dis.pdist(X, metric='euclidean') # 欧几里得距离
```

3、曼哈顿距离

$$\text{dist}(P_1, P_2) = |x_1 - x_2| + |y_1 - y_2|$$

4、余弦相似度

余弦相似度用向量空间中两个向量夹角的余弦值作为衡量两个个体间差异的大小。角余弦越大表示两个向量的夹角越小，夹角余弦越小表示两向量的夹角越大。

5、杰卡德相似系数

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

要用于计算符号度量或布尔值度量的个体间的相似度，只关心个体间共同具有的特征是否一致这个问题。

代码：

导入一般的两两距离计算函数：

```
from sklearn.metrics import pairwise_distances
```

计算距离：dist = pairwise_distances(X,Y,metric='euclidean')

其他的距离指标选择有：cosine, manhattan, jaccard

K-Means聚类算法

流程：

- 1、随机选择k个点作为初始的聚类中心
- 2、对于剩下的点，根据其与聚类中心的距离，将其归入最近的簇
- 3、对每个簇，计算所有点的均值作为新的聚类中心
- 4、重复2,3知道聚类中心不再发生变化

基本概念：

 要得到簇的个数，需要指定K值

 质心：均值，即向量各维取平均即可

 距离的度量：常用欧几里得距离和余弦相似度（先标准化）

 优化目标：
$$\min \sum_{i=1}^K \sum_{x \in C_i} dist(c_i, x)^2$$

在有监督学习中，可以使用聚类“类别id”作为一个新的特征项，然后再用监督学习分别进行学习

聚类评价指标：

- 1、簇内平方和：每个数据点(x_i)距其聚簇中心(C_k)的距离平方和值越小表示聚簇越紧密
- 2、轮廓系数

$$S = \frac{b - a}{\max(a, b)}$$

对每个数据点计算一个轮廓系数：

a = 此数据点到**同簇中**所有其他点的平均距离，**凝聚度**

b = 此数据点到**最近簇中**所有点的平均距离，**分离度**

将所有数据点的轮廓系数取平均值就得到一个总的评分取值在[-1, 1]之间，值越大，聚类效果越好

选择轮廓系数最大的k值，簇内平方和选择拐点

kmeans++

区别是选择质心的方式不一样

1、从数据集中间，任选一个点作为质心。2、把所有样本到质心距离计算出来，选择较远的距离（从离中心点第一远，第二远，选一个距离），作为另外一个质心，**从而解决了在较近的距离选择两个点**

K-Means优缺点

优点：算法简单，易于理解；对球形簇样本聚类效果好，二分k均值等变种算法运行良好，不受初始化问题的影响；算法的复杂度是O（Nkt），对处理大数据集是高效的

缺点：结果与初始质心有关；对噪声和孤立点数据敏感，少量的这些数据就会对平均值产生较大的影响；不能处理非球形簇、不同尺寸和不同密度的簇；在大数据集上收敛较慢

代码

```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++')
```

n_clusters：整型，可选，默认参数值为8，聚类数量。

init：'k-means++'，'random'或者ndarray之一。

代码清单 10-1 K-Means 模型的训练与评估

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
if __name__ == '__main__':
    iris = load_iris()
    model = KMeans(n_clusters=3)
    pred = model.fit_predict(iris.data)
    print(score(pred, iris.target))
```

第12章 卷积神经网络

- 感知机网络：解决线性可分问题
- BP神经网络：解决线性不可分问题
- 深度神经网络：海量图片分类

卷积神经网络大致过程：convolutional layer（卷积）、ReLU layer（非线性映射）、pooling layer（池化）、fully connected layer（全连接）、output（输出）的组合

相比多层感知机网络，卷积神经网络的特点是局部连接，参数共享

全连接与局部连接（权值共享）

卷积神经网络通常由一个输入层和输出层以及多个隐藏层组成。隐藏层包括卷积层、激活层、池化层，以及全连接层。

卷积层是卷积神经网络所特有的一种子结构。一个卷积层包含多个卷积核，卷积核在输入数据上进行卷积计算，从而提取得到特征。一个卷积操作一般由四个超参数组成，即卷积核大小 K （kernel size）、步长 S （stride）、填充 P （padding）以及卷积核数目 C （number of kernels）。具体来说，假设输入的特征大小为 $W' \times H' \times C'$ ，则输出特征的维度 $W \times H \times C$ 为

池化层一般包括两种，一种是平均池化层（Average Pooling），另一种是最大值池化层（Max Pooling）。池化层可以起到保留主要特征，减小一层的参数量和计算量的作用，从而防止过拟合风险。

全连接层一般用于分类网络的最后面，起到类似于“分类器”的作用，将数据的特征映射到样本标记特征。相比卷积层的某一位置的输出仅与上一层中相邻位置有关，全连接层中的每一个神经元都会与前一层的所有神经元有关，因此全连接层的参数量也是很大的。

归一化层包括了 BatchNorm、LayerNorm、InstanceNorm、GroupNorm 等方法，本节仅介绍 BatchNorm。BatchNorm 在批次（batch）的维度上进行归一化，使得深度网络中间卷积的结果也满足正态分布，整个训练过程更快，网络更容易收敛。

过程：

16.3 算法步骤

在介绍算法步骤之前，需要先对多数卷积神经网络所具有的结构元素进行介绍。

(1) 卷积层 (convolutional layer): 卷积神经网络中每个卷积层由若干卷积单元组成，每个卷积单元的参数都是通过反向传播算法最优化得到的。卷积运算的目的是提取输入的不同特征，第一层卷积层可能只能提取一些低级的特征，如边缘、线条和角等层级，更多层的网络能从低级特征中迭代提取更复杂的特征。

(2) 池化层: 池化 (pooling) 是卷积神经网络中另一个重要的概念，它实际上是一种形式的下采样。池化层会不断地减小数据的空间大小，因此参数的数量和计算量也会下降，这在一定程度上也控制了过拟合。通常来说，CNN 的卷积层之间都会周期性地插入池化层。

(3) 全连接层: 全连接层是神经网络的一种基本结构，它的每一个神经元都与上一层的所有神经元相连，用来把前边提取到的特征综合起来。在卷积神经网络中全连接层起到将通过卷积层与池化层学习到的分布式特征表示映射到样本标记空间的作用。卷积神经网络中的全连接层可由卷积操作实现。

(4) 损失函数层 (loss layer): 用于决定训练过程如何来“惩罚”网络的预测结果和真实结果之间的差异，它通常是网络的最后一层。各种不同的损失函数适用于不同类型的任务。例如，softmax 交叉熵损失函数常常被用于在 K 个类别中选出一个类别；sigmoid 交叉熵损失函数常常用于多个独立的二分类问题；欧几里得损失函数常常用于结果取值范围为任意实数的问题。

stride用于设置卷积移动的步幅，卷积特征图尺寸计算公式：

$$F_o = \left\lfloor \frac{F_{in} + 2 \times p - k}{s} \right\rfloor + 1$$

式中 $2p$ 表示对图像的上下左右各填充 p 个像素，式中 $\lfloor \rfloor$ 是向下取整的符号，意思是除不尽的时候向下取整， k 代表卷积核的尺寸， s 就是步长，在卷积网络中特征尺寸的计算可参考我的一篇博文[深度学](#)

CNN采用张量形状 (image_height, image_width, color_channels)

```
image = image.reshape([1, 28, 28, 1]) # 第一个维度是样本个数
```

```
model = models.Sequential() # 实例化
```

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1))) # 卷积层
```

池化

```
model_pool = models.Sequential()
```

```
model_pool.add(layers.MaxPooling2D((2, 2), input_shape=(26, 26, 32))) # 池化操作
```

第15章 BP神经网络

通常指基于误差反向传播算法的多层前向神经网络，BP算法由信号的前向传播和误差的反向传播两个过程组成，主要用于回归预测

前向传播：输入样本从输入层进入网络，经隐含层逐层传递至输出层，如果输出层的实际输出与期望输出不同，则转至误差反向传播过程；如果输出层的实际输出与期望输出相同或网络不再收敛，结束学习算法。

误差反向传播：，输出误差（期望输出与实际输出之差）将按原通路反传计算，通过隐含层反向传播至输入层。在该过程中，在误差将出示盘并个异会分配给各层的神经元，获得各层神经元的误差信号，并将其作为修正各单元权值的根据。整个过程基于梯度下降法实现，不停地调整各层神经元的权值和阈值，使误差信号降低到最低。

缺陷：不能移动，不能变形，运算量大

BP权值更新规则：权值调整量=学习率*局部梯度*上一层输出信号

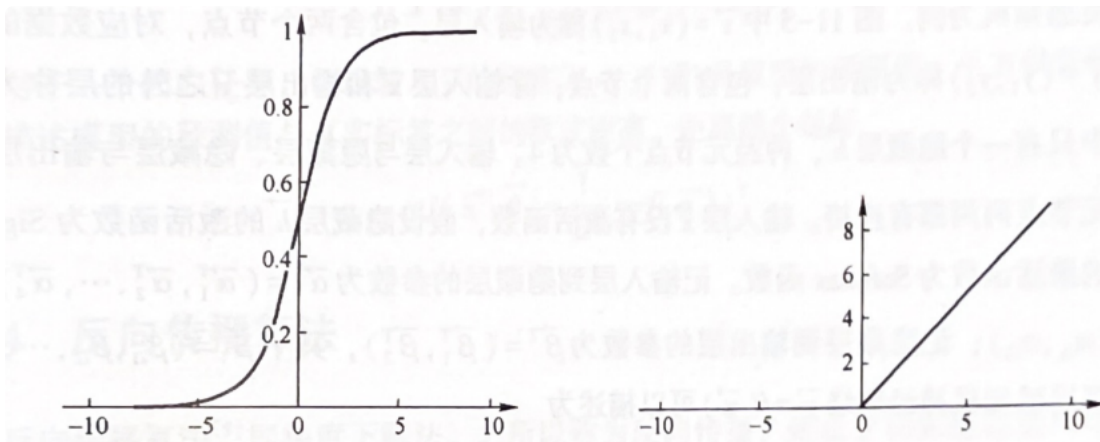
$$a_i = \sum_{j=1}^n w_{ij} x_j \quad y_i = f\left(\sum_{j=1}^n w_{ij} x_j - \theta\right)$$

x_j 为输入信号， f 为传递函数， $w_{i,j}$ 表示与神经元 x_j 连接的权值， y_i 表示输出值， θ 表示阈值

常见激活函数：

(1) sigmoid函数

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$$



(2) relu函数

$$\text{ReLU}(x) = \max(0, x)$$

网络训练过程

输入：训练集数据、学习速率 η

过程：

- 在(0,1)范围内随机初始化网络中所有连接权和阈值
- repeat
 - 根据网络输入和当前参数计算网络输出值 y
 - 计算输出层神经元梯度项 g_j
 - 计算隐层神经元梯度项 e_h
 - 更新连接权值和阈值
- until达到停止条件
- 输出：连接权值和阈值